

MLE5004: Computer systems architecture

III. 80x86 microprocessor architecture.

Contents

3.1	General architecture	1
3.2	CPU Registers	2
3.2.1	General-purpose registers	2
3.2.2	Segment registers	4
3.2.3	EFLAGS Register	4
3.2.4	Instruction pointer	5

In order to understand the instructions of the assembly language you need to understand the architecture of the Intel 80x86 CPU family.

3.1 General architecture

The Intel CPU family is generally classified as a *Von Neumann Architecture Machine*. The generic architecture of a computer is shown in figure 1. The architecture presented by figure 1 does not apply to any specific machine, but is common to all computers. A computer has five main functional units:

1. the arithmetic and logic unit (ALU);
2. the control unit (or sequencer);
3. the storage unit comprising the main memory and cache;
4. the input devices;
5. the output devices.

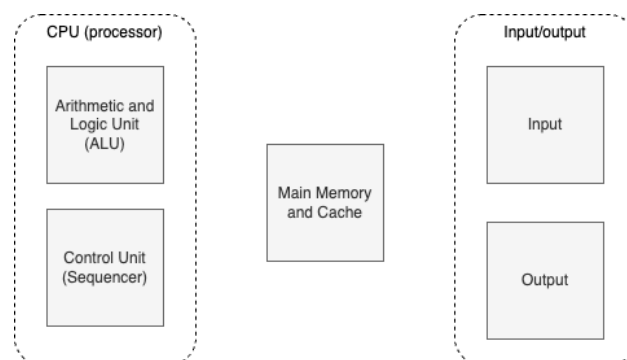


Figure 1: General architecture of a computer.

1. The *ALU* performs arithmetic operations, such as add, subtract, multiply, and divide in fixed-point, decimal, and floating-point number representations; increment and decrement; logical operations, such as AND, OR, NOT, exclusive-OR, exclusive-NOR; shifting operations, such as shift right algebraic (SRA), shift right logical (SRL), shift left algebraic (SLA), and shift left logical (SLL); and rotate operations, such as rotate right and rotate left.
2. The *Control unit*, or control store, is part of the CPU and contains the microprogram, also referred to as microcode. The microprogram is stored in a high-speed memory and accommodates a set of low-level instructions that control the machine's hardware and is machine dependent; that is, it is written for a particular type computer. The instructions in a microprogram represent micro operations that the CPU performs to execute a machine-language instruction. A microprogram is also referred to as *firmware*.
3. The memory unit consists primarily of main memory and cache. It also contains two registers: memory address register and memory data register. The memory address register stores the memory address to which data are written or from which data are read. The memory data register contains the data that is written to memory or read from memory.
- 4, 5. The input/output (I/O) block from Figure 1 incorporates I/O peripheral devices and associated control units. Some examples of I/O device subsystems are disk, keyboard, monitors, plotters, compact disks, and devices attached to a computer by means of a universal serial bus (USB). Many computers also incorporate I/O processors — called channels — to control all data and command transfers between the computer (or memory) and the I/O device.

3.2 CPU Registers

There are a variety of X86 processors: from Intel Corporation, Advanced Micro Devices (AMD), and many other microprocessor companies. Intel processors support the 32-bit and 64-bit architectures. Figure 2 shows a typical X86 processor register set used for most common applications in the IA-32 processor basic execution environment. **There are other registers that are not shown**, such as those used for memory management; single-instruction multiple-data (SIMD); packed floating-point operations; control registers; debug registers; machine check registers; floating-point registers; and the 64-bit registers with R prefix.

The 80x86 CPU registers can be broken down into four categories:

1. general-purpose registers,
2. special-purpose application-accessible registers,
3. segment registers,
4. special-purpose kernel-mode registers.

Segment registers aren't used as much in modern operating systems, as we shall see later on. The special-purpose kernel-mode registers are intended for writing operating systems, debuggers, and other system level tools, which beyond the scope of this course.

3.2.1 General-purpose registers

The eight general-purpose registers can be addressed separately as:

- 8-bit registers (e.g. **AL**),
- 16-bit registers (e.g. **AX**),
- 32-bit registers (e.g. **EAX**).

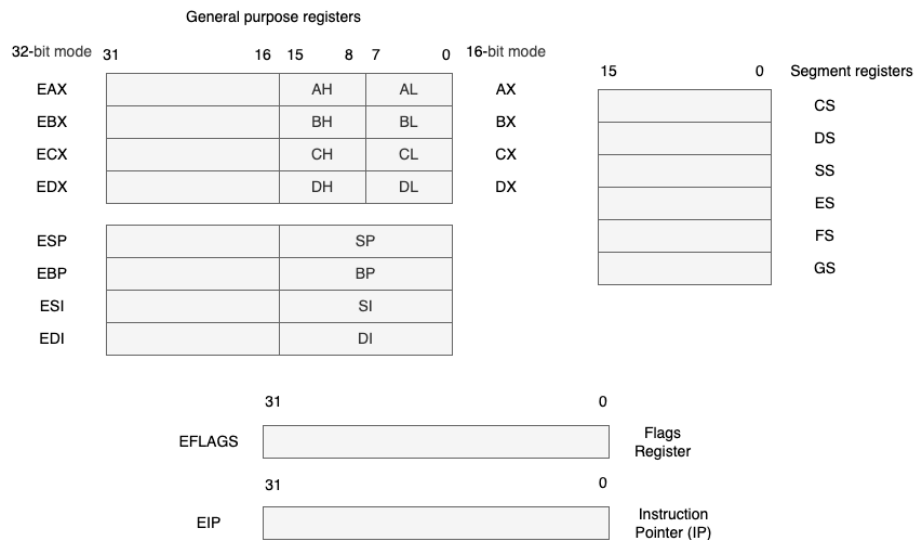


Figure 2: Typical X86 register set for most common applications in the IA-32 basic execution environment.

For example: AL, AX, or EAX. Register AL specifies the low-order eight bits of register AX; register AH specifies the high-order eight bits of register AX; register AX specifies the low-order 16 bits of register EAX.

The general-purpose registers are used for arithmetic operations, logical operations, and for addressing memory. Although they are referred to as general-purpose registers, each register has a specific application.

- Register **EAX** and its constituent registers are used as the **accumulator**; specifically, as the dividend for unsigned and signed divide operations, as the multiplicand for unsigned and signed multiply operations, and for input/output operations. Register AL is also used for the translate (XLAT) instruction.
- Register **EBX** and its constituent registers are used as **index** registers for base addressing and as pointers to data in the data segment (DS). The effective address (EA) is the sum of a displacement value and the contents of one of the EBX registers. When these registers are used, the default location of the operand resides in the current data segment.
- The **ECX** register and its constituent registers are the implied **counters** for certain instructions; for example, to count the number of iterations in a loop. The CL register is also used to indicate the shift amount when shifting or rotating bits left or right. For string operations, the ECX register set denotes the number of operands that are moved from the source string to the destination string.
- The **EDX** register and its associated registers — generally referred to as the data registers — are used for **input/output operations and for multiply and divide operations** for both **unsigned and signed operands**. The multiplicand (destination operand — AL, AX, or EAX) is multiplied by the multiplier (source operand) and the product is stored in the destination operand. The product is stored in register AX, concatenated register pair DX:AX, or concatenated register pair EDX:EAX, depending on the size of the $2n$ -bit result. The resulting quotient is stored in AL, AX, or EAX. The remainder is stored in register AH, DX, or EDX.
- Register **SP** (for 16-bit stack words) or register **ESP** (for 32-bit stack double-words) are **stack pointers** that point to the current top of stack by providing an offset from the stack segment register address to point to the last valid entry in the stack.

- The **BP** and **EBP** registers are **base pointers** that are used to point to data on the stack; for example, parameters pushed onto the stack by one program to be accessed by another program via the stack. The base pointer registers provide an offset from the stack segment register address to point to the required data.
- The **SI** and **ESI** registers are **source index** registers that are used for string operations. They provide an offset in the data segment (DS) that points to the beginning of a source string that is to be transferred to a destination string in the extra segment (ES).
- The **DI** and **EDI** registers are destination index registers that are used for string operations. They provide an offset in the extra segment that points to the beginning address of a destination string that is to be transferred from a source string in the data segment.

3.2.2 Segment registers

When using a segmented memory model in the IA-32 processors, memory is partitioned into independent segments for code, data, and stack. The code segment (CS) register — segment selector — contains the starting address of the computer's code segment. The offset in the instruction pointer (IP or EIP) points to a particular instruction in the code segment. The combination of the segment selector and the offset is the logical address of the next instruction to be executed.

There are four data segments: DS, ES, FS, and GS.

- The data segment (**DS**) register contains the starting address of the program's data segment. This address plus the offset value indicates the logical address of the required data.
- The contents of the extra segment (ES) register plus an offset in the destination index (DI) register points to a destination address in the extra segment for string operations.
- The FS and GS registers are available for different data structures.

3.2.3 EFLAGS Register

The 32-bit EFLAGS register is partitioned into three groups of flags: status flags, control flags, and system flags, as shown in Figure 3.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0	0	0	0	0	0	0	0	0	0	ID	VIP	VIF	AC	VM	RF

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	NT	IOPL		OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

Figure 3: EFLAGS register.

Status flags:

- The **Carry Flag (CF)** is set (it has a value of 1) if there is a carry out of or a borrow into the high-order bit position of an arithmetic operation; otherwise it is reset (it has a value of 0). It is also used with shift and rotate instructions.
- The **Parity Flag (PF)** is set (1) if the low-order byte of the result has an even number of 1s; otherwise it is reset (0).
- The **Adjust Flag (AF or Auxiliary Carry)** is set if there is a carry out of or a borrow into the low-order four bits — used primarily for a binary-coded decimal arithmetic operation; otherwise it is reset.
- The **Zero Flag (ZF)** is set if the result is zero; otherwise it is reset.
- The **Sign Flag (SF)** is set to the value of the high-order bit position, which is the sign bit for signed operands. If the sign flag is 0, then the result is positive; if the sign flag is 1, then the result is negative.

- The **Overflow Flag (OF)** is set if the result of an operation is too large or too small to be contained in the destination operand; that is, the number is out of range for the size of the result. An interrupt-on-overflow (INTO) instruction generates an interrupt if the overflow flag is set.

Control flags:

- The **Direction Flag (DF)** indicates the direction for string operations and is used for the following string instructions: move string (MOVS), compare strings (CMPS), scan string (SCAS), load string (LODS), and store string (STOS). If the direction flag is reset, then this causes the string instruction to auto-increment the index registers (SI and DI); that is, to process strings from left (low address) to right (high address). If the direction flag is set, then this causes the string instruction to auto-decrement the index registers (SI and DI); that is, to process strings from right (high address) to left (low address).

System flags are used to control the operating system and CPU operations:

- if the **Interrupt enable Flag (IF)** is set and an interrupt occurs, then this causes the CPU to transfer control to a memory location specified by an interrupt vector. The CPU can temporarily ignore maskable interrupts, but must respond immediately to non-maskable interrupts, such as non-recoverable hardware errors, time-critical interrupts, or power failure. If the interrupt enable flag is reset, then interrupts are disabled.
- The **Trap Flag (TF)** allows single-step mode for debugging; that is, single instruction execution. The CPU generates an interrupt after each instruction so that the program and results can be examined.
- The **Nested Task (NT)** flag is set if the present task is nested within another task.
- The two bits of the **Input/Output Privilege Level (IOPL)** flag contain the I/O privilege level of the current task and supports multitasking. An IOPL of 00 is the highest level; an IOPL of 11 is the lowest level. The privilege levels are used with the privileged instructions, such as input/output instructions and segment accessibility.
- The **Resume Flag (RF)**, when set, indicates to the CPU to resume debugging. The debug software sets the flag prior to returning to the interrupted program.
- When the **Virtual-8086 Mode (VM)** flag is set, the CPU emulates the program environment of the 8086 processor; when reset, the processor returns to protected mode. Protected mode permits the CPU to use virtual memory, paging, and multi-tasking features.
- The **Alignment Check (AC)** flag is used to check the alignment of memory references. It is set if a word (16 bits) or doubleword (32 bits) is not on a word or doubleword boundary.
- The **Virtual Interrupt Flag (VIF)** is a virtual image of the interrupt flag and is used in combination with the **Virtual Interrupt Pending (VIP)** flag. The VIP flag is set if an interrupt is pending and reset if there is no interrupt pending.

3.2.4 Instruction pointer

The **Instruction Pointer (EIP)** contains the offset for the current code segment that points to the next instruction to be executed. The EIP is updated as shown below for sequential execution, where (EIP) specifies the 'contents of' EIP.

$$\text{EIP} \leftarrow (\text{EIP}) + \text{instruction length}$$

The EIP progresses from one instruction to the next in sequential order, or forwards/backwards a specified number of instructions for jump (JMP), jump on condition (Jcc), return from a procedure/subroutine (RET), or return from interrupt (IRET) instructions.

References

1. A Vancea. Courses, (tank you :))
2. Cavanagh, J. (2013). X86 Assembly Language and C Fundamentals (1st ed.). CRC Press.
<https://doi.org/10.1201/b14582>
3. Randall Hyde. (2020). The Art of Assembly Language (2nd Edition). No Starch Press.
4. Daniel Kusswurm. (2023). Modern X86 Assembly Language Programming: 32-bit, 64-bit, SSE, and AVX. Apress.